

SES :: Project 2

- 1. Introduction
- 2. PWM
 - 2.1. Description
 - 2.2. The code
 - 2.3. Introduced bugs and “safe version”
 - 2.4. Illustration of usage
- 3. Detecting and fixing bugs
 - 3.1. Unit tests for user/password validation
 - 3.2. Issues reported by the clang static analyzer and GCC compiler
 - 3.3. Fuzzing
- 4. Additional challenges
 - 4.1. Design vulnerabilities
 - 4.2. Stack smashing challenge

Author: Eduardo R. B. Marques (<https://www.dcc.fc.up.pt/~edrdo/>), DCC/FCUP

SES Moodle page (<https://moodle2526.up.pt/course/view.php?id=5479>)

1. Introduction

The goal of this project is to test the reliability and security of a small C application — a password manager. To that end, static analyzers, unit tests, fuzzers, and runtime sanitizers will be employed.

This project accounts for 4 / 20 of the final grade.

Deadline

June 5

Submission instructions

You will have to submit a PDF report and the modified source code. The following guidelines should be followed for the PDF report:

- it can be written in Portuguese or English;
- it should be organized in sections and have 10 pages at most, including any figures, code listings and tables;
- references to external sources (e.g., web pages, books, articles) should be included in a final references section;
- figures, tables and code listings should be numbered and referenced in the text;
- the font size for normal text should be 10 pt.

A Moodle submission form will be available closer to the deadline.

Setup instructions

Download the ZIP archive with the code from Moodle, and follow the instructions in this document. You should be able to work on the project using Google Cloud Shell or a Linux machine.

2. PWM

2.1. Description

pwm is a rudimentary (and insecure!) password file manager, with a format somewhat similar to but simpler than the /etc/shadow (<https://linux.die.net/man/5/shadow>) file on Linux systems. An example password file is as follows:

```
admin:68ce6c40:f699d9208839b227a050ce73c8558662
steve:d28d18a9:3b7ac81dfd7bcb735cd0bc596f821afc
linus:02279492:d57aef52733f7ea2fa9f08cd7c44c0ed
edrdo:64507481:8ba7a666b19c26b302db038d093eaae7
```

Each line has 3 fields:

- the user ID;
- a 4-byte salt obtained from `/dev/urandom`;
- and an MD5 checksum generated from the salt and the user's password.

The admin user that appears on the first line (in principle!) is special: subsequent openings of the file require confirmation of this user's password, and this user cannot be deleted (supposedly!).

2.2. The code

The source code is located in the top directory.

Before you compile it, make sure the BSD library functions are installed:

```
sudo apt install libbsd-dev
```

Then, to compile the code, run:

```
make clean all
```

The code is compiled by GCC in debug mode, allowing you to use the [gdb](https://www.gnu.org/software/gdb/) (<https://www.gnu.org/software/gdb/>) debugger if desired, and instrumented for fault detection via the [AddressSanitizer](https://clang.llvm.org/docs/AddressSanitizer.html) (<https://clang.llvm.org/docs/AddressSanitizer.html>) sanitizer.

The code is organized into several files:

File	Description
<code>pwm.h</code>	header file with type and function declarations
<code>pwm.c</code>	program's entry point (the <code>main</code> function)
<code>commands.c</code>	handling of commands entered by the user
<code>core.c</code>	core functions for password file manipulation
<code>validation.c</code>	validation functions for checking valid usernames and passwords accepted by the system
<code>utils.c</code>	utility functions such as input handling, salt generation, and MD5 hash computation for passwords
<code>md5.c</code>	RSA code for MD5 checksum computation, slightly adapted

A small set of C library functions is used. The main ones used are for:

- string manipulation: `strcmp`, `strcpy`, `strchr`, ...;
- byte sequence copy/comparison: `memcpy`, `memcmp`;
- dynamic memory allocation: `malloc`, `free`, `strdup`;
- I/O: `gets`, `fgets`, `fopen`, `fclose`, `fprintf`,

2.3. Introduced bugs and “safe version”

A number of faults (source code defects) have been deliberately introduced into `pwm`, some of which constitute security vulnerabilities, including:

- logical programming errors in the validation of usernames and passwords, to be identified through unit tests (see separate section);
- use of insecure C library functions;
- improper heap memory management leading to use-after-free, double-free, and memory leak cases;
- other logical programming errors.

A “safe version” without deliberately introduced faults is the binary program `pwmsafe`. It is provided without debug information and you may run it in Google Cloud Shell to understand what is the expected behavior of `pwm`.

2.4. Illustration of usage

Executing the program interactively, you may issue commands. The `help` command lists the available options. You can also execute `help <cmd>` to get the description of a specific command.

▼ Illustration

```
PWM command (e.g. 'help') [1]: help
>> Command: 'help' [ ]
Available commands:
  init <file> <admin-pass> : initialize PWM data (without saving to
file)
  open <file> <admin-pass>: open PWM file supplying admin password.
  save : save PWM data to file.
  clear: clear all PWM data in memory (all changes lost)
  add <user> <pass> : add user/password pair
  delete <user> : delete user
  update <user> <pass> : update password for user
  check <user> <pass> : check that password is correct for user
  list PWM entries
  quit : quits the program
  help [<cmd>]: displays this message or help for specific command
<< 'help' -- success.
PWM command (e.g. 'help') [2]: help add
>> Command: 'help' [ 'add' ]
  add <user> <pass> : add user/password pair
<< 'help' -- success.
```

The following is a sample run of pwmsafe
that produces a password file like the one at the beginning of this section.

▼ Example execution

```
PWM command (e.g. 'help') [1]: init passfile.txt Qses1920!
>> Command: 'init' [ 'passfile.txt' 'Qses1920!' ]
<< 'init' -- success.
PWM command (e.g. 'help') [2]: add bill Gates!0
>> Command: 'add' [ 'bill' 'Gates!0' ]
<< 'add' -- success.
PWM command (e.g. 'help') [3]: add steve sb0J.Mac
>> Command: 'add' [ 'steve' 'sb0J.Mac' ]
<< 'add' -- success.
PWM command (e.g. 'help') [4]: add linus T0rvalds
>> Command: 'add' [ 'linus' 'T0rvalds' ]
<< 'add' -- success.
PWM command (e.g. 'help') [5]: add edrdo inv_pass
>> Command: 'add' [ 'edrdo' 'inv_pass' ]
PWM error! Invalid password 'inv_pass'!
<< 'add' -- failure (error code 8) !
PWM command (e.g. 'help') [6]: add edrdo Zxy199.
>> Command: 'add' [ 'edrdo' 'Zxy199.' ]
<< 'add' -- success.
PWM command (e.g. 'help') [7]: add edrdo ZZZZ1x9
>> Command: 'add' [ 'edrdo' 'ZZZZ1x9' ]
PWM error! User 'edrdo' already exists!
<< 'add' -- failure (error code 6) !
PWM command (e.g. 'help') [8]: update edrdo ZZZZ1x9
>> Command: 'update' [ 'edrdo' 'ZZZZ1x9' ]
<< 'update' -- success.
PWM command (e.g. 'help') [9]: delete bill
>> Command: 'delete' [ 'bill' ]
<< 'delete' -- success.
PWM command (e.g. 'help') [10]: save
>> Command: 'save' [ ]
<< 'save' -- success.
PWM command (e.g. 'help') [11]: quit
>> Command: 'quit' [ ]
<< 'quit' -- success.
```

A subsequent run may use the previously generated file.

▼ Using the same file

```
PWM command (e.g. 'help') [1]: open passfile.txt Qses1920!
>> Command: 'open' [ 'passfile.txt' 'Qses1920!' ]
<< 'open' -- success.
PWM command (e.g. 'help') [2]: list
>> Command: 'list' [ ]
admin:68ce6c40:f699d9208839b227a050ce73c8558662
steve:d28d18a9:3b7ac81dfd7bcb735cd0bc596f821afc
linus:02279492:d57aef52733f7ea2fa9f08cd7c44c0ed
edrdo:64507481:8ba7a666b19c26b302db038d093eaae7
4 users
<< 'list' -- success.
PWM command (e.g. 'help') [3]: clear
>> Command: 'clear' [ ]
<< 'clear' -- success.
PWM command (e.g. 'help') [4]: quit
>> Command: 'quit' [ ]
<< 'quit' -- success.
```

Finally, note that a file containing commands may also be passed as an argument.

▼ Execution passing command file as argument

```
$ cat example.txt
init passfile.txt Qses1920!
add bill Gates!0
add steve sb0J.Mac
add linus T0rvalds
add edrdo inv_pass
add edrdo Zxy199.
add edrdo ZZZZ1x9
update edrdo ZZZZ1x9
delete bill
list
save
quit

$ ./pwm example.txt
...
```

3. Detecting and fixing bugs

The aim is to try to uncover and fix bugs in different ways:

1. We first make use of unit tests to uncover bugs in rules for user name and password validation (pwm/validation.c file).
2. We then deal with issues reported by the clang static analyzer.
3. We finally make use of fuzzing to uncover more bugs.

3.1. Unit tests for user/password validation

The code

See pwm/validation.c for the validation code in the functions pwm_is_valid_user and pwm_is_valid_password.

The rules

A username should be considered valid by pwm_is_valid_user if:

1. It is between 4 and 10 characters long.
2. It consists *only* of lowercase letters: 'a' to 'z'.

A password should be considered valid by `pwm_is_valid_password` if (the constraints are not particularly strong!):

1. It is between 6 and 12 characters long.
2. It contains **at least one** lowercase letter — a to z.
3. It contains **at least one** uppercase letter — A to Z.
4. It contains **at least one** digit — 0 to 9.
5. It contains **at most one** of the following punctuation characters: . , : ! ?.
6. It does not contain any subsequence found in a (small) blacklist (see source code).

Running the tests

In the `gtest` directory you will find `testuser.cpp` and `testpass.cpp`, which encode unit tests for `pwm_is_valid_user` and `pwm_is_valid_password`, respectively. The tests are structured using the Google Test (<https://github.com/abseil/googletest/blob/master/googletest/docs/primer.md>) framework, and code coverage reports are generated by the gcov (<https://gcc.gnu.org/onlinedocs/gcc/Gcov.html>) tool.

Running `make` will (re)compile (if necessary) and execute the tests, producing:

1. Details about the test run, identifying tests that passed and those that failed.
2. A file `validation.c.gcov` with the test coverage analysis.

▼ Test execution illustration

```
$ cd gtest
$ make
[=====] Running 4 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 4 tests from test_user
[ RUN      ] test_user.short_length
[          OK ] test_user.short_length (0 ms)
[ RUN      ] test_user.invalid_char
testuser.cpp:11: Failure
Expected equality of these values:
    PWM_INVALID_USER_ID
      Which is: 7
    pwm_is_valid_user("@rui")
      Which is: 0
[ FAILED   ] test_user.invalid_char (0 ms)
[ RUN      ] test_user.valid_user
testuser.cpp:15: Failure
Expected equality of these values:
    PWM_OK
      Which is: 0
    pwm_is_valid_user("ruiruiruir")
      Which is: 7
[ FAILED   ] test_user.valid_user (0 ms)
[ RUN      ] test_user.valid_user2
[          OK ] test_user.valid_user2 (0 ms)
[-----] 4 tests from test_user (0 ms total)

[-----] Global test environment tear-down
[=====] 4 tests from 1 test suite ran. (1 ms total)
[ PASSED   ] 2 tests.
[ FAILED   ] 2 tests, listed below:
[ FAILED   ] test_user.invalid_char
[ FAILED   ] test_user.valid_user

2 FAILED TESTS
[=====] Running 4 tests from 1 test suite.
[-----] Global test environment set-up.
```

```
[-----] 4 tests from test_pass
[ RUN      ] test_pass.invalid_pass_length5
[          OK ] test_pass.invalid_pass_length5 (0 ms)
[ RUN      ] test_pass.valid_pass_length6
testpass.cpp:11: Failure
Expected equality of these values:
    PWM_OK
    Which is: 0
    pwm_is_valid_password("Aa9za1")
    Which is: 8
[  FAILED   ] test_pass.valid_pass_length6 (2 ms)
[ RUN      ] test_pass.invvalid_pass_with_two_punct
testpass.cpp:15: Failure
Expected equality of these values:
    PWM_INVALID_PASSWORD
    Which is: 8
    pwm_is_valid_password("Aa9za1!!")
    Which is: 0
[  FAILED   ] test_pass.invvalid_pass_with_two_punct (1 ms)
[ RUN      ] test_pass.blacklisted_pass1
testpass.cpp:19: Failure
Expected equality of these values:
    PWM_INVALID_PASSWORD
    Which is: 8
    pwm_is_valid_password("A123zad!")
    Which is: 0
[  FAILED   ] test_pass.blacklisted_pass1 (1 ms)
[-----] 4 tests from test_pass (6 ms total)

[-----] Global test environment tear-down
[=====] 4 tests from 1 test suite ran. (7 ms total)
[  PASSED   ] 1 test.
[  FAILED   ] 3 tests, listed below:
[  FAILED   ] test_pass.valid_pass_length6
[  FAILED   ] test_pass.invvalid_pass_with_two_punct
[  FAILED   ] test_pass.blacklisted_pass1
```

```
3 FAILED TESTS
File '../validation.c'
Lines executed:83.64% of 55
Branches executed:90.91% of 44
Taken at least once:65.91% of 44
No calls
../validation.c:creating 'validation.c.gcov'
```

Writing new tests and fixing bugs

The goal is to write tests that identify bugs in the source code of the user and password validation functions, while also achieving 100% branch coverage. Describe in your report the defects you found and the code changes you made to ensure correct behavior of `pwm_is_valid_user` and `pwm_is_valid_password`.

In summary:

1. Inspect the errors produced by the failing tests in `testuser.cpp` and `testpass.cpp` and investigate what problems may exist in the code (also make sure the tests themselves are correctly written!). Fix the code.
2. Write tests to achieve 100% branch coverage in `pwm_is_valid_user` and `pwm_is_valid_password`. You should add tests to improve coverage and possibly discover more bugs.
3. In the report, explain what bugs have been fixed and what extra tests you had to include to attain 100% branch coverage.

3.2. Issues reported by the clang static analyzer and GCC compiler

The code may be analyzed by the clang static analyzer. To avoid installation of clang, a report of the issues can be found [here \(clang-static-analysis/index.html\)](https://clang.llvm.org/static-analyzer/index.html). Furthermore the compilation using GCC reports a number of warnings (most of which are also reported by the clang static analyzer).

▼ GCC compilation warnings

```

commands.c: In function 'pwm_handle_command':
commands.c:71:6: warning: format not a string literal and no format
arguments [-Wformat-security]
    71 |         printf(command_args[j]);
        |         ^~~~~~
----
core.c: In function 'pwm_alloc':
core.c:44:5: warning: pointer 'pwm' used after 'free' [-Wuse-after-
free]
    44 |         free(pwm);
        |         ^~~~~~
core.c:41:5: note: call to 'free' here
    41 |         free(pwm);
        |         ^~~~~~
---
... /pwm/commands.c:261:(.text+0x16de): warning: the `gets' function
is dangerous and should not be used.

```

For the reported issues do the following:

1. Explain in the report why the code is sensitive in terms of security and/or reliability, and how during execution there may be a problem causing a sanitizer crash and/or some other type of error.
2. Fix the code to correct the issue. Mention what you did in the report.

3.3. Fuzzing

Can fuzzing help us discover more vulnerabilities? Consider the use of black-box fuzzing using the [blab](https://gitlab.com/akihe/blab) (<https://gitlab.com/akihe/blab>) and [radamsa](https://gitlab.com/akihe/radamsa) (<https://gitlab.com/akihe/radamsa>) tools. Inside the fuzzing directory you will find the source code of both tools and some auxiliary files.

Compiling blab and radamsa

In the fuzzing directory execute make to compile blab and radamsa.

► Compilation of blab and radamsa

Generating input files using blab and radamsa

For illustration, two scripts are provided — `gblab.sh` and `gradamsa.sh` — along with the `grammar.blab` grammar for blab. Adapt these resources as you see fit. In particular, to make the examples generated by blab more representative and larger in size, you should extend/adapt the grammar in `grammar.blab`.

▼ Example usage for blab

```
$ ./gblab.sh
Running blab - see inputs dir for generated files
$ ls inputs
blab1.txt  blab2.txt  blab4.txt  blab6.txt  blab8.txt
blab10.txt blab3.txt  blab5.txt  blab7.txt  blab9.txt
<details>
$ cat inputs/blab1.txt
help delete
init password.txt Qses1920
add edward ZZ34567890129
add admin 123456789012
add charlesf abcdef.e
add charlesf y12!3456
add charles Zz3456789012
list
clear
quit
```

▼ Example usage for radamsa

```
$ ./gradamsa.sh seeds/blab*.txt
Transforming given files using radamsa
seeds/blab1.txt.rad generated
seeds/blab10.txt.rad generated
seeds/blab2.txt.rad generated
seeds/blab3.txt.rad generated
seeds/blab4.txt.rad generated
seeds/blab5.txt.rad generated
seeds/blab6.txt.rad generated
seeds/blab7.txt.rad generated
seeds/blab8.txt.rad generated
seeds/blab9.txt.rad generated
$ diff inputs/blab1.txt inputs/blab1.txt.rad
2,4c2,4
< init password.txt Qses1920
< add edward ZZ34567890129
< add admin 123456789012
---
> init password.txt Qses-1040149
> add edward ZZ34570882392
> add admin 129
```

Running the fuzzing script

The `fuzzpwm.sh` script can be used to execute `pwm` for a number of input files, detect program crashes for the executions, and gather coverage information.

1. It takes one or more input files as arguments, for instance `./fuzzpwm.sh inputs/blab1.txt` or `./fuzzpwm.sh inputs/*`
2. It runs `pwm` for each input `f`, warning of possible program crash. For file `f` output files can be found in `outputs/f.out` and `outputs/f.err`.
3. At the end, coverage information will be derived accounting for all executions. Coverage information files can be found in the `coverage` directory.

▼ Example execution

```
$ ./fuzzpwm.sh inputs/blab1.txt inputs/blab2.txt
--- Cleaning up previous execution files ...
--- Fuzzing ...
Executing pwm for input file inputs/blab1.txt
Possible program crash for inputs/blab1.txt - see
outputs/blab1.txt.out for more information
Executing pwm for input file inputs/blab2.txt
Possible program crash for inputs/blab2.txt - see
outputs/blab2.txt.out for more information
--- Done ... 2 possible program crashes in 2 executions
--- Deriving coverage information ...
File 'commands.c'
Lines executed:66.44% of 149
Branches executed:83.87% of 62
Taken at least once:56.45% of 62
Calls executed:66.67% of 54
Creating 'commands.c.gcov'

File 'core.c'
Lines executed:27.96% of 186
Branches executed:29.27% of 82
Taken at least once:18.29% of 82
Calls executed:22.39% of 67
Creating 'core.c.gcov'

File 'md5.c'
Lines executed:98.41% of 126
Branches executed:100.00% of 16
Taken at least once:81.25% of 16
Calls executed:92.31% of 13
Creating 'md5.c.gcov'

File 'pwm.c'
Lines executed:75.00% of 8
Branches executed:100.00% of 4
Taken at least once:50.00% of 4
Calls executed:66.67% of 3
```



```
Creating 'pwm.c.gcov'
```

```
File 'utils.c'
```

```
Lines executed:65.62% of 64
```

```
Branches executed:57.14% of 28
```

```
Taken at least once:39.29% of 28
```

```
Calls executed:100.00% of 15
```

```
Creating 'utils.c.gcov'
```

```
File 'validation.c'
```

```
Lines executed:88.64% of 44
```

```
Branches executed:90.91% of 44
```

```
Taken at least once:68.18% of 44
```

```
No calls
```

```
Creating 'validation.c.gcov'
```

```
Lines executed:62.74% of 577
```

```
coverage/commands.c.gcov coverage/md5.c.gcov coverage/utils.c.gcov
```

```
coverage/core.c.gcov coverage/pwm.c.gcov
```

```
coverage/validation.c.gcov
```

The coverage information can help identify which parts of the code are not being reached by the fuzzer, and motivate the manual addition or automatic generation of a larger number and more meaningful inputs.

Identifying and fixing vulnerabilities

Using `fuzzpwm.sh`, several vulnerabilities in `pwm` may manifest. For each such problematic input, you should attempt to identify the problem in the source code and fix it. In your report:

- identify each of the issues;
- describe what type of inputs cause it to manifest (not the entire input file, but the specific sequence that leads to a crash);
- and explain how you fixed the code.

4. Additional challenges

4.1. Design vulnerabilities

We are considering a very simple and vulnerable password manager. Consider the following problems:

1. Note that an adversary (assuming they have write access to the password file) could remove or alter the position of the `admin` user's entry in a password file without `pwm` noticing. Explain why and how an adversary could exploit this vulnerability. Modify the `pwm_open` function to fix this design flaw.
2. Similarly, an adversary with limited technical knowledge could duplicate a line in the file and change only the username to obtain a password equivalent to that of the original user (who may have a known password). This happens because the MD5 checksum only takes into account the salt and the password, but not the username. Generalize `pwm_hash_password` in `utils.c` to address this, and modify the program's logic at other points as needed.
3. Even after fixing the issues above, nothing prevents arbitrary tampering with the file in order to freely manipulate users and passwords, or to corrupt it. Do you have any proposals for addressing this problem? Explain in the report the general principles of what you propose. You are not required to implement your proposal (though you can also do it!).

4.2. Stack smashing challenge

Preserve or restore the `gets` calls and the format string vulnerability detected by the static analysis (in **3.2**). Explain how these vulnerabilities can be exploited in stack-smashing attacks to improperly obtain a shell. Additionally, you should try to implement the attacks concretely.

For this purpose, you may find several files and utilities in the `pwm/ss` directory similar to those used in the lab class.

- To compile `pwm` and other auxiliary programs in the `ss` directory, use:

```
make clean all [OPTIONS]
```

where the optional `OPTIONS` arguments can enable different types of protection for the `pwm` executable, **all disabled by default**:

- `CANARIES=1`: enables the use of stack canaries;
- `NOXSTACK=1`: disables executable code on the stack;
- `PIE=1`: enables PIE code;
- In a Linux machine, you should also disable ASLR, for instance using

```
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
```

In case you have no root access to do that, use the `nr.sh` script to disable ASLR, for example:

```
./nr.sh ./pwm
```

- A simple `hello` program is also provided in case you wish to test stack-smashing.